

Requirements Technical Analysis, Affected Modules & Effort Timeline

System: **Yellow Markets (YMStore)** | Scope Clarifications & New Feature Assessment

Document Version: 1.0	Project: YMStore Core & Portals	Assessment Date: September 2026
Scope Type: SOW Alignment + New Requests	Platforms: Admin, Merchant, Storefront, Web API	Total Estimated Effort: ~270 Hours

1. Executive Summary & Architecture Overview

This technical document provides a comprehensive impact assessment for the **5 SOW Scope Alignment items** and **4 New Requests (BCOD / MUYP Discussion)**. For each requirement, the affected architectural layers, impacted controllers/models/views, database alterations, risk level, and engineering effort estimates are detailed. Development is split into **Phase 1: SOW Discrepancy Alignments (130 hrs)** and **Phase 2: New Requests (140 hrs)**, spanning a compressed **6-week delivery roadmap**.

Item #	Requirement Title	Type	Primary Affected Modules	Est. Hours
SOW-1	Resolution Process (Helpdesk Escalation)	SOW Scope	Admin, Merchant, Helpdesk DB	36 hrs
SOW-2	YM Delivery (Weight, Crates & 60kg Limit)	SOW Scope	Storefront Cart, Web API, Cataloging	40 hrs
SOW-4	Social Sharing (Instagram / LinkedIn)	SOW Scope	Product Detail Views, Share APIs	14 hrs
SOW-5	Category Filtering (Main, Sub, Child)	SOW Scope	Catalog Navigation, AJAX Filters	24 hrs
SOW-6	SEO (Sitemap.xml, robots.txt, Open Graph)	SOW Scope	Storefront Header, SEO Controllers	16 hrs
REQ-1	Daily Deals / Flash Sales Configuration	New Request	Merchant Catalog, Promotions, Crons	36 hrs
REQ-2	Deal Recognition & Urgency Badges	New Request	Listing Cards, Search, Trending, PDP	26 hrs
REQ-3	Shopper Notification System	New Request	Storefront Header, Notification DB	32 hrs
REQ-4	Add-on & Subscription Invoicing / Records	New Request	Admin/Merchant Addons & Subs, PDF	46 hrs
TOTAL	Consolidated Architecture Scope (9 Modules)	All	Full-Stack Impact	270 hrs

2. SOW Scope Alignments — Technical Deep-Dive

Item 1: Resolution Process (Helpdesk Escalation to Merchant)

Current Architecture Gap: Shoppers open tickets in the storefront Helpdesk (`help\_desk` table). Admin reviews them via `CustomerController.php` (`shopper\_help\_desk\_list.php`), but there is no mechanism for Admin to escalate a resolution request directly to the concerned merchant (`publisher\_id`), attach order items, or initiate a dedicated Admin-Merchant mediation chat.

Solution Architecture:

- Add `escalation\_status`, `assigned\_merchant\_id`, and `resolution\_type` to `help\_desk` table.
- Create a mediation conversation model (`help\_desk\_merchant\_conversations`) connecting Admin and Merchant.
- Build Merchant Resolution Inbox in Merchant Portal (`merchant/application/controllers/UserController.php`).
- Trigger real-time notifications to Merchant and Admin on resolution thread updates.

Affected Files:

- admin/application/controllers/CustomerController.php
- admin/application/views/shopper_help_desk_list.php, help_desk_conversation.php
- merchant/application/controllers/UserController.php
- merchant/application/views/help_desk/ (New Resolution Management View)
- application/models/CommonModel.php, admin/application/models/CommonModel.php

Effort Estimate: 36 Hours (Analysis: 4h, Backend & DB: 14h, Admin/Merchant UI: 12h, QA: 6h)

Item 2: YM Delivery — Weight Intervals, Crate Sizes & 60kg Order Blocking

Current Architecture Gap: Shipping currently uses `shipping\_methods\_charges` based on historical tiers. Orders exceeding 60 Kg are not halted, allowing 120 Kg shipments to be accepted at incorrect flat rates.

Solution Architecture:

- **Mandatory Product Attributes:** Implement mandatory `weight` (kg) and `crate\_size` (Small, Medium, Large) in Product Cataloging for all products.
- **Interval Calculation:** Rate formula = $\text{ceil}(\text{total_weight} / 20) * 300$ MUR for YM Delivery.
- **Hard Limit Validation:** Enforce maximum total shipment weight = **60 Kg**. In `CartController.php`, `CheckoutController.php`, and `webapi/src/Model/DbCheckout.php`, if weight > 60 Kg: disable checkout, block order placement, and show alert prompting the shopper to split order into multiple shipments.
- **ES Order Volume:** Store calculated volume and weight into `b2b\_orders` for logistics fulfillment.

Affected Files:

- admin/application/controllers/Sellerproduct.php, merchant/application/controllers/Sellerproduct.php
- admin/application/views/products/, merchant/application/views/seller/products/
- application/controllers/CartController.php, application/controllers/CheckoutController.php
- application/views/cart/cart.php, application/views/checkout/checkout.php
- webapi/src/Model/DbCheckout.php (method `getShippingChargesByQuoteWeight`)
- webapi/src/Routes/webshop/Checkout.php, webapi/src/Routes/webshop/Cart.php

Effort Estimate: 40 Hours (Product Setup: 10h, Pricing Engine: 12h, Cart/Checkout Blocking: 10h, QA: 8h)

Item 4: Social Sharing — Instagram & LinkedIn Integration

Technical Evaluation: Instagram does not provide direct query-parameter URL sharing endpoints (unlike Facebook/LinkedIn). For Instagram: implement Web Share API (native mobile sharing directly to Instagram Stories/Direct) with fallback to high-conversion 'Copy Link to Instagram'. For LinkedIn: implement official Share-Offsite URL scheme.

Affected Files:

- application/views/product/product_details.php (Product detail page share toolbar)
- application/views/components/fast_view_product_details.php (Quick view modal share)
- public/js/social_share.js (New responsive social sharing handler)

Effort Estimate: 14 Hours (Design & Integration: 8h, Multi-device Testing: 6h)

Item 5: Category Filtering — Main, Sub & Child Category Multi-Level Navigation

Current Architecture Gap: `category\_menu.php` only links to child categories once expanded. Shoppers cannot multi-select or filter products across top/mid categories with checkboxes.

Solution Architecture: Introduce checkbox selection for Main Category, Level 1 Sub-Category, and Level 2 Child Category in `catalog\_filters.php` and `category\_menu.php`. When parent is checked, all children are queried seamlessly via AJAX catalog filters.

Affected Files:

- application/views/components/category_menu.php, catalog_filters.php
- application/controllers/ProductsController.php
- webapi/src/Routes/webshop/Product.php, webapi/src/Routes/webshop/Search.php
- public/js/catalog_filter.js

Effort Estimate: 24 Hours (UI/UX Tree: 8h, Filter Controller & Web API: 10h, Cross-browser QA: 6h)

Item 6: SEO Architecture — Dynamic Sitemap, Root robots.txt & Open Graph

Current Architecture Gap: No active `robots.txt` at storefront root. `sitemap.xml` route is missing. `application/views/common/header.php` lacks `og:title`, `og:image`, `og:description`, `og:url` tags.

Solution Architecture:

- Create root `robots.txt` pointing to `https://yellowmarkets.com/sitemap.xml`.
- Implement dynamic XML Sitemap generator controller mapping all active products, categories, shops, and CMS URLs.
- Inject dynamic Open Graph meta tags into `header.php` using product images, category banners, and site fallbacks.

Affected Files:

- robots.txt (Root creation)
- application/config/routes.php (`sitemap.xml` route)
- application/controllers/SitemapController.php (New dynamic controller)
- application/views/common/header.php (Open Graph tag injection)

Effort Estimate: 16 Hours (Sitemap Generator: 6h, OG Tag Integration: 6h, SEO Validator QA: 4h)

3. New Requests (BCOD / MUYP Discussion) — Technical Deep-Dive

Request 1: Daily Deals & Flash Sales — Unified Scheduling & Single-Step Configuration

Current Architecture Gap: Merchants must first navigate to discount setups, then separately edit products to bind them. Scheduled calendar start dates do not automate deal activation.

Solution Architecture:

- Consolidate discount configuration, deal quota verification, and product binding into a single interactive modal inside Product Management.
- Add `deal\_start\_datetime` and `deal\_end\_datetime` to product promotions.
- Implement automated scheduling cron (`CronController::process\_scheduled\_deals()`) that activates offers on the exact Calendar Start Date and auto-reverts upon expiration.

Affected Files:

- merchant/application/controllers/Sellerproduct.php, admin/application/controllers/Sellerproduct.php
- merchant/application/views/seller/products/edit_product.php
- admin/application/controllers/CronController.php
- merchant/application/models/Subscription_model.php (Deal quota verification)

Effort Estimate: 36 Hours (UI Consolidation: 14h, Scheduler Cron & DB: 12h, Validation & QA: 10h)

Request 2: Daily Deals & Flash Sales — Storefront Recognition Badges & Urgency Counters

Current Architecture Gap: Storefront product listings (search, categories, trending, new arrivals) render standard product cards without distinguishing between normal items, Daily Deals, or Flash Sales.

Solution Architecture:

- **Daily Deals (Time-Sensitive):** Display distinct Amber Deal Ribbon, discount %, and real-time countdown timer.
- **Flash Sales (Stock-Sensitive):** Display Crimson Flash Badge, remaining inventory counter (e.g. 'Only 4 left!'), and dynamic progress bar.
- Update listing queries to return active deal flags in a single query without N+1 performance bottlenecks.

Affected Files:

- application/views/components/product_listing_page_item.php
- application/views/components/trending_products.php, newarrival_products.php, home_categorys_products.php
- application/views/product/product_details.php (PDP Urgency Box)
- webapi/src/Model/DbProductFeature.php, webapi/src/Routes/webshop/Product.php

Effort Estimate: 26 Hours (Badging Component: 10h, Query Optimization: 8h, Mobile/Desktop QA: 8h)

Request 3: Shopper Notification System (Orders, Status, Helpdesk & Resolution)

Current Architecture Gap: The existing `notifications` table only supports `recipient\_type = 'admin'` and `merchant`. Shoppers do not have in-app notification center, unread bell count, or event triggers on order status changes.

Solution Architecture:

- Support `recipient\_type = 'shopper'` with `recipient\_id = customer\_id` in `notifications` table.
- Build Shopper Notification Bell & Dropdown in storefront navigation header (`common/header.php`).
- Hook automated notification dispatchers for: Order Confirmation, Shipped/Delivered events, Helpdesk Replies, and Resolution Actions.
- Dedicated Shopper Notifications center (`/customer/notifications`).

Affected Files:

- application/views/common/header.php (Bell UI & real-time counter)
- application/controllers/MyProfileController.php (`notifications()`, `mark\_as\_read()`) endpoints)
- application/views/myprofile/notifications.php (Full notification inbox view)
- merchant/application/controllers/B2BOrdersController.php (Delivery status notification trigger)
- admin/application/controllers/CustomerController.php (Helpdesk notification trigger)

Effort Estimate: 32 Hours (DB & Event Hooks: 12h, Header & Inbox UI: 12h, QA: 8h)

Request 4: Add-on & Subscription Purchase Management, Invoicing & Notifications

Current Architecture Gap: Add-on purchases do not generate official tax invoices, lack purchase notifications, and have insufficient visibility in Admin and Merchant portals.

Solution Architecture:

- **Admin Panel:** Complete Add-on and Subscription Purchase Management table (Order Ref, Merchant, Add-on/Plan, Amount, VAT, Payment Method, Status, Invoice PDF download). Real-time admin notification on new purchases.
- **Merchant Panel:** 'My Add-ons' and 'My Subscriptions' overview showing active quotas, expiration dates, transaction history, and instant invoice PDF downloads.
- **Automated Invoicing Engine:** Standardized PDF invoice generation for every Add-on and Subscription purchase matching Yellow Markets branding.

Affected Files:

- admin/application/controllers/Addons.php, AddonManageService.php
- admin/application/views/addons/list.php, manage_subscription_list.php
- merchant/application/controllers/Addons.php, UserController.php
- merchant/application/views/addons/index.php, manage_subscription_list.php
- admin/application/views/invoice/addon_invoice.php, subscription_invoice.php
- merchant/application/views/invoice/addon_invoice.php, subscription_invoice.php

Effort Estimate: 46 Hours (Invoice PDF Engine: 16h, Admin Modules: 12h, Merchant Modules: 10h, QA: 8h)

4. Implementation Timeline & Milestone Roadmap

The project is structured into a **compressed 6-week delivery roadmap** across 4 development sprints and a final UAT / Production rollout phase. Development streams are parallelized to meet the 6-week target.

Sprint / Phase	Focus Modules & Deliverables	Duration	Target Timeline	Allocation
Sprint 1 SOW Core Fixes	• SOW-2: YM Delivery (Weight attributes, 20kg interval, 60kg cart blocking) • SOW-6: SEO (Dynamic sitemap.xml, root robots.txt, Open Graph meta)	56 Hours	Week 1 – Week 1.5	Full-stack Dev + QA
Sprint 2 SOW UX & Resolution	• SOW-1: Helpdesk Shopper Resolution escalation & Admin-Merchant thread • SOW-4: Social Sharing (Instagram/LinkedIn) • SOW-5: Multi-level Category Filtering (Main/Sub/Child)	74 Hours	Week 1.5 – Week 3	Full-stack Dev + QA
Sprint 3 Promotions & Badges	• REQ-1: Daily Deals / Flash Sales single-flow config & start-date scheduler • REQ-2: Storefront Recognition badges (Countdown timer & Stock counters)	62 Hours	Week 3 – Week 4.5	Backend + Frontend Dev
Sprint 4 Invoicing & Alerts	• REQ-3: Shopper Notification Center (Order, Status, Helpdesk alerts) • REQ-4: Add-on & Subscription Invoicing, Purchase History & Notifications	78 Hours	Week 4.5 – Week 5.5	Full-stack Dev + QA
UAT & Release Final Deployment	• Staging End-to-end regression, Client UAT Signoff, Production rollout	Buffer / QA	Week 5.5 – Week 6	Tech Lead & DevOps

5. Technical Recommendations & Risk Mitigation

- 1. Database Migration Strategy:** Several new columns are required (`help\_desk.escalated\_to\_merchant`, `b2b\_orders.shipping\_weight`, `notifications.recipient\_type = 'shopper')`. All migrations must be scripted via backwards-compatible SQL migrations without downtime.
- 2. 60kg Weight Limit Enforcement:** Must be validated at both frontend (JS warning) and backend (`CartController` & Web API `DbCheckout`). Any attempt to submit an order over 60kg must be strictly rejected at server level.
- 3. Social Sharing Consistency:** Instagram web sharing should use the HTML5 Navigator Share API for mobile devices, seamlessly passing product image and URL, with an automated clipboard fallback for desktop browsers.
- 4. Unified Deal Scheduling Cron:** A single cron job running every 15 minutes (`CronController::process\_scheduled\_deals`) will handle both activation and expiry to prevent server load spikes.

Prepared For: BCOD & MUYP Discussion	Document Type: Scope Assessment & Estimation
Status: Proposal Ready for MUYP Discussion	Target Kickoff: Immediate upon signoff